TASK 12 – Day-4 Smart-Meter Design Challenge

A. Impact Analysis

In the updated system, smart meters send readings in watt-hours every 15 minutes. Electricity used between 6 PM and 10 PM costs 20% more than electricity used during other hours. The tariff rates must also be changeable without recompiling the program.

Each house produces:

35,040 readings per year.

Scale	Annual readings
4,000 houses	140,160,000
60,000 houses	2,102,400,000
800,000 houses	28,032,000,000

A 32-bit signed int is not large enough to store the district's annual reading count. Therefore, a 64-bit counter and wider energy totals should be used.

Hard-coded tariff macros are no longer suitable because the rates must be changed while keeping the same executable.

B. Evaluation of the Original Design:

The original design uses integer paise to avoid rounding errors, which is still a good choice. However, 16-bit meter readings are not suitable for storing large cumulative energy totals.
The original design also uses fixed tariff values. These need to be replaced with values read from a configuration file so that tariff changes can be made without recompiling the program.

C. Revised Design:

The revised program can read the base tariff in paise per kWh from a configuration file. It then applies a multiplier of 120 during peak hours and 100 during other hours.

C program:

```
#include <stdio.h>
#include <stdint.h>
int64_t calculate_charge(uint64_t wh, int peak,
int64_t rate)
{
int64_t multiplier = peak ? 120 : 100;
return (int64_t)((wh * (uint64_t)rate *
(uint64_t)multiplier + 50000ULL)
/ 100000ULL);
}
```

```
int main(void)
{
printf("Charge: %lld paise\n",
(long long)calculate_charge(1500, 1, 225));
return 0;
}
```

Expected output:

Charge: 405 paise

The charge is ₹4.05 for 1,500 Wh at a rate of 225 paise per kWh during peak hours. The example rounds the charge to the nearest paisa. In a real system, the multiplication should also be checked to prevent overflow.

D. Evidence-Based Defence:

The revised design continues to store money as integer paise to keep the calculations accurate. It uses 64-bit counters to handle the large number of smart-meter readings. Tariff rates can be updated through a configuration file, and the program applies the higher rate during peak hours.

Processing N readings takes O(N) time. Saving readings as they arrive also avoids keeping all the readings for an entire year in memory.

Conclusion:

The billing program uses suitable data types, stores money in paise, keeps tariff values in one place, and checks user input. A wide integer variable helps prevent overflow when calculating the total collection.

For the smart-meter system, the design needs to support larger reading counts, tariff rates that can be changed without recompiling, peak-hour pricing, and clear rules for rounding and preventing overflow.